

FHIRScan

Technische Dokumentation

Automatisiertes FHIR R4B Dataset Profiling | Version 1.0 | April 2026 | BHC | Carla Schlüter

Inhalt

1 Überblick

FHIRScan ist ein Python-Skript, das FHIR R4B-Datensätze automatisiert analysiert und ein vollständiges statistisches Metadatenprofil erstellt. Es liest JSON- und NDJSON-Dateien aus einem Eingabeverzeichnis, traversiert alle enthaltenen Ressourcen feldweise und exportiert die Ergebnisse als CSV- und Textdateien.

Das Skript überträgt dabei **keine Patientendaten** — es arbeitet ausschließlich auf Metaebene (Struktur, Häufigkeiten, Typen). Es eignet sich für den Einsatz direkt an der Datenquelle innerhalb der sicheren Umgebung eines DIZ.

Kernfunktionen auf einen Blick

- Feldvollständigkeit (Presence Rate / Missingness) je Ressourcentyp und Feld
- Top-N häufigste Werte inkl. Häufigkeit, min/max für numerische Felder
- Automatische FHIR-Typerkennung (CodeableConcept, Reference, Period, Quantity, ...)
- Kardinalitäten: Ressourcen je Patient und je Encounter
- Referenzintegritätsprüfung: verwaiste FHIR-Referenzen explizit ausgewiesen
- Zeitliche Verteilung und Strukturtiefenanalyse je Ressourcentyp
- HyperLogLog-Schätzung für Felder mit hoher Kardinalität (RAM-effizient)

2 Voraussetzungen & Installation

2.1 Systemvoraussetzungen

- Python 3.9 oder neuer
- Keine externen Python-Bibliotheken erforderlich — ausschließlich Standardbibliothek (`json`, `csv`, `statistics`, `hashlib`, `re`)
- Ausreichend RAM abhängig von Datensatzgröße (Faustregel: ~500 MB für 1 Mio. Ressourcen)
- Lesezugriff auf das FHIR-Eingabeverzeichnis, Schreibzugriff auf das Ausgabeverzeichnis

2.2 Installation

Da FHIRScan keine externen Abhängigkeiten hat, ist keine Installation im klassischen Sinne erforderlich. Das Skript kann direkt ausgeführt werden:

```
# Repository klonen oder main.py direkt herunterladen
git clone https://github.com/example/fhirscan.git
cd fhirscan

# Direkt ausführen (kein pip install nötig)
python main.py <input_dir> <output_dir>
```

2.3 Empfohlene Umgebung

- Ausführung innerhalb der vertrauenswürdigen Zone des DIZ (keine Daten verlassen die Umgebung)
- Für große Datensätze (> 5 Mio. Ressourcen): Ausführung auf einem dedizierten Server empfohlen
- Ausgabeverzeichnis sollte auf einem lokalen Dateisystem liegen — Netzlaufwerke verlangsamen den CSV-Export

3 Verwendung

3.1 Aufruf

```
python main.py <input_dir> <output_dir> [Optionen]

# Beispiele:
python main.py /data/fhir /output/profil
python main.py /data/fhir /output/profil --top-n 50
python main.py /data/fhir /output/profil --no-relations --quiet
```

3.2 Parameter

Alle verfügbaren Parameter und ihre Standardwerte:

Parameter	Standard	Beschreibung
input_dir	—	Pflichtargument: Verzeichnis mit FHIR-Dateien (rekursiv)
output_dir	—	Pflichtargument: Ausgabeverzeichnis (wird angelegt wenn nötig)
--top-n N	20	Anzahl der häufigsten Werte, die je Feld gespeichert werden
--hll-precision N	14	HyperLogLog-Präzision (4–16); höher = genauer, mehr RAM
--hll-threshold N	10000	Ab diesem Schwellwert wird auf HLL-Schätzung umgeschaltet
--no-inline-extensions	—	Extension-URLs nicht als Pfadsegmente inlinen
--no-relations	—	Relationale Analyse überspringen (schneller)

`--quiet / -q``-`

Keine Fortschrittsausgabe auf stdout

3.3 Unterstützte Eingabeformate

- FHIR Bundle (JSON): Einzelne Bundles mit `resourceType: Bundle` und `entry[]`-Array
- NDJSON: Eine Ressource oder ein Bundle pro Zeile (Newline-Delimited JSON)
- Beliebig tief verschachtelte Verzeichnisstrukturen werden rekursiv durchsucht
- Deduplizierung: Ressourcen mit identischer (`resourceType`, `id`)-Kombination werden nur einmal gezählt

3.4 Ausgabe-Ordnerstruktur

FHIRScan legt im angegebenen Ausgabeverzeichnis automatisch einen timestamped Unterordner an:

```
output_dir/
├── 20260417_09_30/           ← Timestamp: YYYYMMDD_HH_MM
│   ├── Patient_fields.csv
│   ├── Encounter_fields.csv
│   ├── Observation_fields.csv
│   ├── _summary.csv
│   ├── _summary_details.csv
│   ├── _cardinality.csv
│   ├── _data_quality.csv
│   ├── _reference_integrity.txt
│   ├── _temporal_analysis.txt
│   ├── _structural_analysis.txt
│   └── _errors.txt           ← Nur wenn Fehler aufgetreten sind
```

4 Ausgabedateien im Detail

4.1 Übersicht aller Ausgabedateien

Dateiname	Typ	Inhalt
Patient_fields.csv	CSV	Feldprofil: Presence Rate, Top-N-Werte, FHIR-Typ je Feld
Encounter_fields.csv	CSV	Analoges Profil für Encounter-Ressourcen
<ResourceType>_fields.csv	CSV	Ein File je Ressourcentyp im Datensatz
_summary.csv	CSV	Übersicht: Ressourcentypen, Feldanzahl, Konsistenz
_summary_details.csv	CSV	Feldliste mit Presence-Kategorie je Ressourcentyp
_cardinality.csv	CSV	Kardinalitäten: Ressourcen je Patient / Encounter
_data_quality.csv	CSV	Verlinkungsraten Patient- und Encounter-Referenzen

_reference_integrity.txt	TXT	Bericht: valide, verwaiste und unbekannte Referenzen
_temporal_analysis.txt	TXT	Zeitliche Verteilung, Datumsreichweiten je Ressourcentyp
_structural_analysis.txt	TXT	Strukturtiefe und Array-Größen je Ressourcentyp
_errors.txt	TXT	Laufzeitfehler und Parse-Warnungen (bei Auftreten)

4.2 Spaltenreferenz: <ResourceType>_fields.csv

Jede ressourcentypspezifische CSV-Datei enthält eine Zeile pro Feldpfad mit folgenden Spalten:

Spalte	Bedeutung
field_path	Vollständiger JSON-Pfad inkl. Array-Markierung, z.B. Observation.code.coding[].system
resources_with_field	Anzahl Ressourcen, in denen dieses Feld mindestens einmal vorkommt
total_resources	Gesamtanzahl Ressourcen dieses Typs im Datensatz
presence_rate	Anteil Ressourcen mit Feld (0.00–1.00)
missing_rate	1 – presence_rate
value_count	Gesamtanzahl gesehener Werte inkl. Duplikate
unique_count	Anzahl distinct values (exakt oder HLL-Schätzung)
unique_count_approximate	TRUE wenn HLL-Schätzung aktiv (> hll-threshold distinct values)
detected_python_types	Beobachtete Python-Typen: string, integer, boolean, null, object, array
detected_fhir_type	Erkannter FHIR R4B-Datentyp (CodeableConcept, Reference, Period, ...)
type_inconsistency	TRUE wenn dasselbe Feld mit mehreren inkompatiblen Typen befüllt ist
top_values	JSON-Array der N häufigsten Werte mit Häufigkeit, z.B. [{"male",512},{"female",488}]

4.3 Feldpfad-Notation

FHIRScan verwendet eine eigene Pfadnotation, die Arrays und Extensions lesbar darstellt:

- `Observation.code.coding[].system` — Array-Elemente werden mit `[]` markiert; alle Items teilen denselben Pfad
- `Patient.extension[http://example.org/birthPlace].valueString` — Extension-URLs werden bei aktivem `--inline-extensions` (Standard) direkt als Pfadsegment eingebettet
- Skalare Felder: direkte Pfadangabe ohne Suffix, z.B. `Patient.gender` oder `Encounter.status`

4.4 _cardinality.csv — Kardinalitätsanalyse

Diese Datei enthält für jeden Ressourcentyp, der Patient- oder Encounter-Referenzen enthält, statistische Kennzahlen über die Verteilung der Ressourcen je Anker:

- Beispiel: Wie viele Observations hat ein Patient im Schnitt? Min, Max, Median, Standardabweichung, P25/P75/P90/P99

- Basis ist immer die Anzahl Ressourcen pro eindeutigem Anker (Patient-ID oder Encounter-ID), nicht die Gesamtzahl
- `coverage_rate`: Anteil der bekannten Patienten/Encounters, die mindestens eine Ressource dieses Typs haben

4.5 `_reference_integrity.txt` — Referenzprüfung

FHIRScan prüft alle FHIR-Referenzen im Datensatz gegen die tatsächlich vorhandenen Resource-IDs:

- **Valide Referenz**: Ziel-ID im Datensatz vorhanden (gleicher ResourceType)
- **Verwaiste Referenz (Orphan)**: Ziel-ID nicht gefunden — möglicher Integritätsfehler oder externer Verweis
- **Unbekannter Typ**: Referenz konnte nicht geparkt werden (z.B. ungewöhnliches Format)
- Die Datei listet bis zu 20 Beispiele verwaister Referenzen mit Quelle, Pfad und Zielangabe

5 Technische Architektur

5.1 Verarbeitungspipeline

FHIRScan ist als single-pass Pipeline implementiert — jede Ressource wird genau einmal geladen und verarbeitet, ohne sie vollständig im Speicher zu halten:

1. Dateisystem-Scan: Rekursives Glob über *.json und *.ndjson
2. Streaming-Parse: `safe_json_load()` gibt Objekte als Generator zurück — kein vollständiges Laden großer Dateien
3. Bundle-Extraktion: Ressourcen aus `Bundle.entry[].resource` werden einzeln weitergegeben
4. Deduplizierung: (resourceType, id)-Set verhindert Mehrfachzählung bei geteilten Ressourcen
5. Feldtraversierung: `traverse_resource()` traversiert rekursiv und emittiert (path, value, type, is_first)-Tupel
6. Aggregation: Statistiken werden inkrementell akkumuliert — kein Batch-Processing erforderlich
7. Relationale Analyse: Parallel zur Feldanalyse werden Referenzen, Zeitstempel und Tiefen extrahiert
8. Export: Einmaliger Schreibvorgang aller CSV/TXT-Dateien am Ende

5.2 HyperLogLog für Kardinalitätsschätzung

Für Felder mit sehr hoher Anzahl unterschiedlicher Werte (z.B. Ressourcen-IDs, freie Texte) ist eine exakte Zählung aller distinct Values speicherintensiv. FHIRScan verwendet daher eine zweistufige Strategie:

- Bis zum Schwellwert (Standard: 10.000 distinct values): exakte Zählung via Python `set`
- Ab dem Schwellwert: automatischer Wechsel auf **HyperLogLog (HLL)** — probabilistischer Algorithmus mit ~2% Fehler bei < 1 KB Speicherverbrauch pro Feld
- Die Ausgabe enthält `unique_count_approximate: TRUE` wenn HLL aktiv ist
- HLL-Präzision (4–16) steuerbar via `--hll-precision` — Präzision 14 entspricht 16.384 Registern (~16 KB, <2% Fehler)

5.3 FHIR R4B Typerkennung

FHIRScan erkennt FHIR-Datentypen durch einen zweilagigen Mechanismus, ohne gegen ein Schema zu validieren:

- **Layer 1 — Strukturelle Fingerprints:** JSON-Objekte werden anhand ihrer Schlüsselkombination identifiziert (z.B. `{coding: [...]}` → CodeableConcept, `{reference: "..."} → Reference)`
- **Layer 1b — String-Narrowing:** Regex-basierte Erkennung von FHIR-Primitiven: instant, dateTime, date, uuid, oid, canonical, uri, base64Binary, code, string
- **Layer 2 — Feldnamen-Lookup:** Normative Feldnamen aus der R4B-Spezifikation (z.B. `telecom` → ContactPoint, `period` → Period) als Fallback
- Polymorphe Felder (value[x]-Pattern) werden durch Suffix-Matching erkannt: `valueQuantity` → Quantity, `onsetAge` → Age

5.4 Konfigurierbare Parameter (programmatisch)

Für den direkten Python-Aufruf kann ein `ProfilerConfig`-Objekt übergeben werden:

```
from main import ProfilerConfig, run_profiler
from pathlib import Path

config = ProfilerConfig(
    input_dir = Path('/data/fhir'),
    output_dir = Path('/output'),
    top_values_limit = 50,          # Mehr Top-Werte
    hll_precision = 16,           # Höhere HLL-Genauigkeit
    hll_threshold = 50000,        # Längere exakte Zählung
    inline_extensions = True,     # Extension-URLs in Pfaden
    analyze_relations = True,     # Relationale Analyse aktiv
    verbose = False,             # Keine stdout-Ausgabe
)

result = run_profiler(config.input_dir, config.output_dir, config)
```

6 Performance & Skalierung

6.1 Laufzeitcharakteristik

- Single-threaded — keine Parallelisierung, einfach nachvollziehbar und debugbar
- Speicher: $O(F \times R)$ wobei F = Anzahl distinct Feldpfade, R = Anzahl Ressourcentypen — nicht $O(N)$ über Ressourcenanzahl
- Typische Laufzeiten (Referenzwerte, 8-Core Server, SSD):
 - 100.000 Ressourcen: ca. 30–60 Sekunden
 - 1 Mio. Ressourcen: ca. 5–10 Minuten
 - 10 Mio. Ressourcen: ca. 45–90 Minuten

6.2 RAM-Verbrauch

- Dominiert durch `value_counter` (Counter aller Werte für Top-N-Berechnung) und `exact_values` (Set bis HLL-Schwellwert)

- Faustregel: ~200–500 MB für einen typischen DIZ-Datensatz mit 5 Ressourcentypen und 1 Mio. Ressourcen
- Bei Speicherproblemen: `--top-n 5` und `--hll-threshold 1000` reduzieren den Footprint erheblich

6.3 Bekannte Einschränkungen

- Keine Streaming-Ausgabe: alle Ergebnisse werden erst am Ende geschrieben
- Kein Multi-Threading: bei sehr großen Datensätzen empfiehlt sich Sharding nach Ressourcentyp
- Nested Extensions mit mehr als 50 Hierarchieebenen werden abgeschnitten (`max_depth = 50`)
- Wertstrings werden bei > 200 Zeichen truncated (konfigurierbar via `max_value_length`)

7 Fehlerbehandlung & Logging

FHIRScan fährt bei Einzelfehlern nicht ab, sondern sammelt Fehler und schreibt sie am Ende in `_errors.txt`. Folgende Fehlertypen werden abgefangen:

- **JSON-Parse-Fehler:** Ungültige Zeilen in NDJSON-Dateien werden übersprungen, Warnung wird geloggt
- **Dateizugriffsfehler:** Dateien ohne Leseberechtigung werden übersprungen
- **Ressourcen ohne resourceType:** Werden gezählt aber nicht weiter verarbeitet
- **Verarbeitungsfehler je Ressource:** Caught und geloggt mit ResourceType/ID, übrige Ressourcen laufen weiter

Wenn `_errors.txt` im Ausgabeverzeichnis existiert, sollte sie nach einem Lauf geprüft werden. Eine leere Fehlerdatei wird nicht erstellt.

8 Sicherheit & Datenschutz

Wichtiger Hinweis zum Datenschutz

FHIRScan gibt in den Ausgabedateien die häufigsten Feldwerte aus. Bei Feldern mit geringer Kardinalität (z.B. gender, status, category) sind das aggregierte Kategorienwerte ohne Personenbezug. Bei Freitext-Feldern (z.B. Notizen, Adressen) können jedoch konkrete Werte in `top_values` erscheinen. Das `--top-n 0` Flag deaktiviert Top-Werte vollständig, falls dies für bestimmte Felder datenschutzrechtlich erforderlich ist.

- FHIRScan liest ausschließlich lokal — es werden keine Netzwerkverbindungen aufgebaut
- Keine Pseudonymisierung oder Anonymisierung wird vorgenommen — das Skript setzt voraus, dass der Ausführungsort bereits datenschutzkonform ist
- Die Ausgabedateien sollten wie die Quelldaten behandelt und nicht ohne weiteres weitergegeben werden

9 Quickstart — Schritt für Schritt

9. FHIR-Daten in einem zugänglichen Verzeichnis bereitstellen (z.B. /data/fhir-export/)
10. Sicherstellen, dass Python 3.9+ installiert ist: `python --version`
11. `main.py` in das Arbeitsverzeichnis legen oder Pfad verwenden
12. Ausgabeverzeichnis anlegen oder FHIRScan anlegen lassen: `mkdir /output/fhirscan`
13. Ersten Scan starten:

```
python main.py /data/fhir-export /output/fhirscan

# Erwartete Ausgabe:
# =====
# FHIR Dataset Profiler - Enhanced with Relational Analysis
# =====
# Scanning directory: /data/fhir-export
# Found 42 files (1.24 GB)
# ...
# Processing Complete!
#   Files processed: 42
#   Total resources: 1,240,000
#   Resource types: 12
#   Unique patients: 5,200
#   Unique encounters: 38,400
```

14. Ergebnisse im Ausgabeverzeichnis prüfen — Unterordner mit Timestamp enthält alle CSVs
15. `_summary.csv` als ersten Einstieg öffnen: gibt Überblick über alle Ressourcentypen
16. Ggf. `_errors.txt` prüfen, falls Warnungen ausgegeben wurden